Searching Algorithms 🔍

Searching algorithms are used to find a specific element or a set of elements from a given data structure. The efficiency of a search algorithm is typically measured by its time complexity, which is dependent on the type of data structure and whether the data is sorted.

1. Linear Search

Linear Search is the most straightforward searching algorithm. It sequentially checks each element in the data structure until a match is found or the end is reached. It does not require the data to be sorted.

- **How it works**:

1. Start at the beginning of the data structure.
2. Compare the target element with the current element.
3. If they match, return the index.
4. If they don't match, move to the next element and repeat.

- **Time Complexity**: **O(n)**, as in the worst case, it has to traverse the entire list.
- **Use Case**: Small, unsorted datasets.

2. Binary Search

Binary Search is a highly efficient algorithm for searching a **sorted** list. It works by repeatedly dividing the search interval in half.

- **How it works**:

1. Compare the target element with the middle element of the sorted list.
2. If they match, the search is successful.
3. If the target is smaller, the search continues in the left half of the list.
4. If the target is larger, the search continues in the right half of the list.
5. This process continues until the element is found or the search interval becomes empty.

- **Time Complexity**: **O(log n)**, which is significantly faster than linear search for large datasets.
- **Prerequisite**: The data must be sorted.

Sorting Algorithms 🔢

Sorting algorithms arrange elements in a list or array in a specific order (e.g., ascending or descending). The efficiency of these algorithms is crucial for many applications, including databases and search engines.

1. Bubble Sort

Bubble Sort is a simple sorting algorithm that repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order. The pass through the list is repeated until the list is sorted.

- **How it works**:

1. Start from the beginning of the list.
2. Compare the first two elements. If the first is greater than the second, swap them.
3. Move to the next pair of elements and repeat.
4. After one pass, the largest element "bubbles" to the end.
5. Repeat this process for the remaining elements until no more swaps are needed.

- **Time Complexity**: **$O(n^2)$**, which makes it inefficient for large datasets.

2. Selection Sort

Selection Sort divides the list into a sorted and an unsorted part. It repeatedly selects the smallest (or largest) element from the unsorted part and moves it to the sorted part.

- **How it works**:

1. Find the minimum element in the unsorted list.
2. Swap it with the first element of the unsorted list.
3. Consider the first element as sorted and repeat the process for the remaining unsorted portion.

- **Time Complexity**: **$O(n^2)$**, regardless of the initial order of the elements.

3. Insertion Sort

Insertion Sort is a simple sorting algorithm that builds the final sorted array one item at a time. It is efficient for small datasets or data that is already substantially sorted.

- **How it works**:

1. The first element is considered sorted.
2. Take the next element from the unsorted part and insert it into its correct position in the sorted part by shifting the larger elements.
3. Repeat until the entire list is sorted.

- **Time Complexity**: **$O(n^2)$** in the worst case, but **$O(n)$** in the best case (when the array is already sorted).

4. Merge Sort

Merge Sort is a highly efficient, divide-and-conquer algorithm. It recursively divides the list into two halves until it reaches single-element lists, and then it merges the sorted halves back together.

- **How it works**:

1. **Divide**: Recursively break the list down into sublists until each sublist contains a single element.

2. **Conquer**: Merge the sublists back together in a sorted order.

- **Time Complexity**: **O(n log n)**, which is highly efficient and stable.
- **Use Case**: Large datasets where efficiency is a priority.

5. Quick Sort

Quick Sort is another popular divide-and-conquer sorting algorithm. It picks an element as a **pivot** and partitions the other elements into two sub-arrays according to whether they are smaller or larger than the pivot.

- **How it works**:

1. Choose a pivot element from the list.
2. Reorder the list so that all elements with values less than the pivot come before it, while all elements with values greater than the pivot come after it.
3. Recursively apply the same process to the sub-arrays of smaller and greater elements.

- **Time Complexity**: **O(n log n)** on average, but **O(n^2)** in the worst case (e.g., when the pivot selection is poor and the list is already sorted).